MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

Data Structures and Algorithms (DSA)

LUMINAR
TECHNOLAB

What is Data Structures and Algorithms (DSA)?

Data Structures and Algorithms (DSA) form the foundation of computer science and programming.

Mastering DSA helps in :

1. writing efficient code
2. optimizing performance
3. solving complex computational problems.

Data Structures

These are ways to store and organize data efficiently for easier access and modification.

Examples :

- *Arrays*
- *Linked Lists*
- *Stacks*
- *Queues*
- *Trees*
- *Graphs*
- *Hash Tables*
- *Heaps.*

Algorithms

These are step-by-step procedures or formulas for solving problems.

Common algorithms include:

- *Sorting (Bubble Sort, Merge Sort)*
- *Searching (Binary Search)*
- *Graph Algorithms (Dijkstra's Algorithm)*

Java Collection Framework

How is Java Collection Framework Related to DSA?

The Java Collection Framework (JCF) is a set of classes and interfaces that implement various data structures and algorithms in Java.

Ie. It provides built-in implementations of common data structures, saving developers from writing them from scratch.

Java Collections use efficient algorithms internally, such as hashing, tree traversal, and sorting, making DSA concepts practically applicable in Java development.

Java Collection Framework	implements	DSA
List Interface • ArrayList • LinkedList	→	• dynamic arrays • linked lists
Set Interface • HashSet • TreeSet	→	• hash tables • trees.
Queue Interface • PriorityQueue • LinkedList	→	queue-based data structures.
Map Interface • HashMap • TreeMap	→	• key-value pair storage with hashing • trees.

Need for Collection Framework

Although we can use an array to store a group of elements of the same type (either primitives or objects). The array, however, does not support so-called dynamic allocation - it has a fixed length which cannot be changed once allocated.

Furthermore, array is a simple linear structure. *Many applications may require more complex data structure.*

Before Collection Framework (or before JDK 1.2) was introduced, the standard methods for grouping Java objects (or collections) were Array or Vector or Hashtable.

All three of these collections had no common interface. Ie. if we want to access elements of array, vector or Hashtable. All these three have different methods and syntax for accessing members

Demo Program

```
package collectionpack;

import java.util.Hashtable;
import java.util.Vector;

public class BeforeJCF {

    public static void main(String[] args) {
```

```java
// Array instance creation requires [],

int arr[] = new int[] { 1, 2, 3, 4 };


// while Vector and hastable require ()
Vector v = new Vector();


Hashtable<Integer, String> h = new Hashtable();


// Vector element insertion requires addElement(),
v.addElement(1);
v.addElement(2);


// Hashtable element insertion requires put()

h.put(1, “luminar");
h.put(2, “java");
```

```java
    // Accessing Array elements are using []
    System.out.println(arr[0]);


    // vector elements using elementAt()
    System.out.println(v.elementAt(0));


    // hashtable elements using get()
    System.out.println(h.get(1));

    }

}
```

What is Collection or Collection Framework in Java?

Collection in java is a **framework** that **provides** an **architecture** to **store and manipulate the group of objects.**

or

Dynamically allocated data structures are supported in a unified architecture called the Collection Framework.

A framework, by definition, is a set of interfaces that force you to adopt some design practices.

Here , Each item in a collection is called an element.

A well-designed framework can improve your productivity and provide ease of maintenance.

The Collection interface (java.util.Collection) and Map interface (java.util.Map) are two main root interfaces of Java collection classes.

The Java Collection Framework package (java.util) contains:

1. A set of interfaces
2. Implementation classes
3. Algorithms (such as sorting and searching)

1. Interfaces: Interface in Java refers to the abstract data types. (The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.)

2. Classes: Classes in Java are the implementation of the collection interface.

3. Algorithm: Algorithm refers to the methods which are used to perform operations such as searching and sorting, on objects that implement collection interfaces. Algorithms are polymorphic in nature as the same method can be used to take many forms or you can say perform different implementations of the Java collection interface.

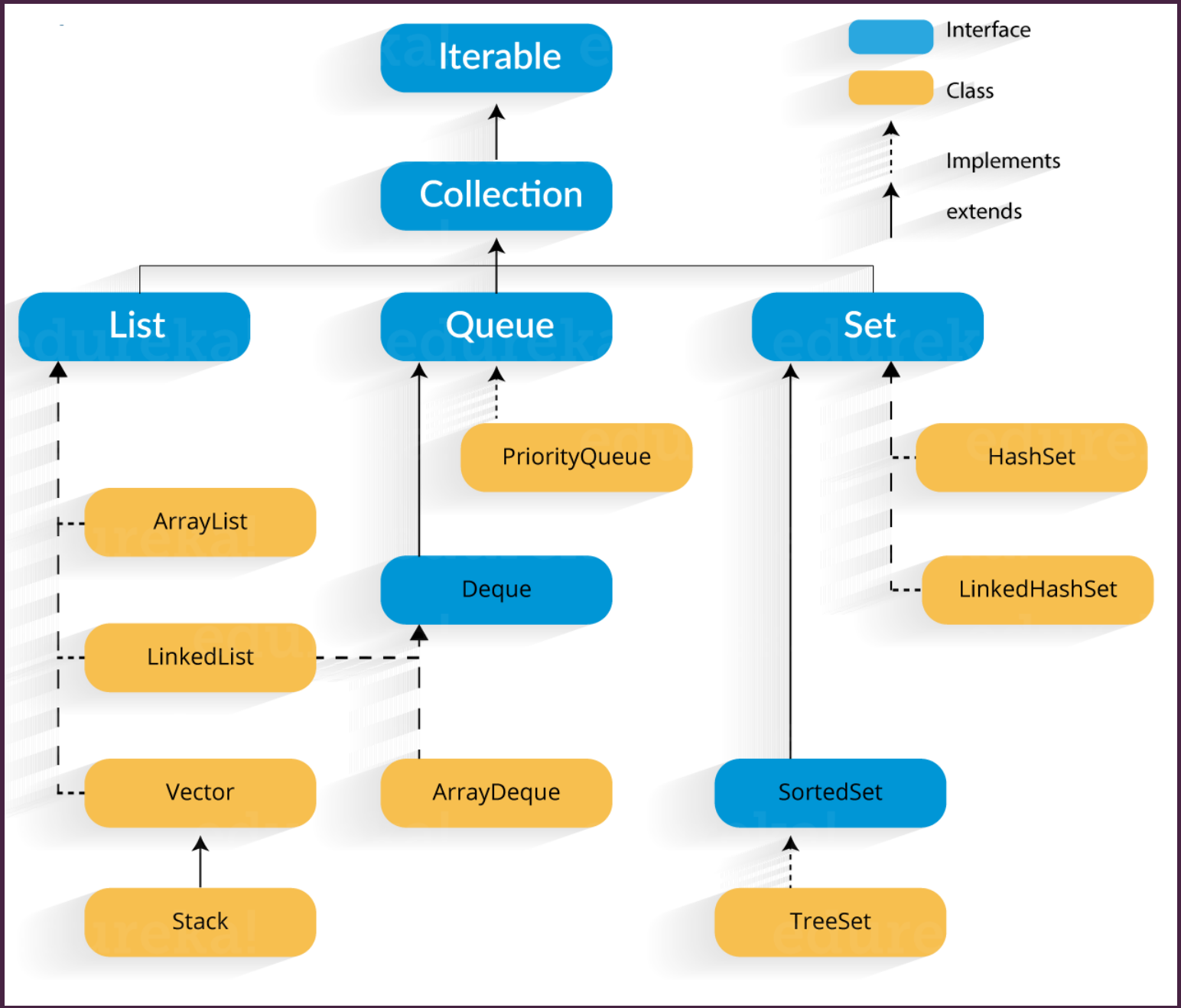
Advantages of Collection Framework

1. Consistent API : The API has basic set of interfaces like Collection, Set, List, or Map. All those classes (such as ArrayList, LinkedList, Vector etc) which implements, these interfaces have some common set of methods.

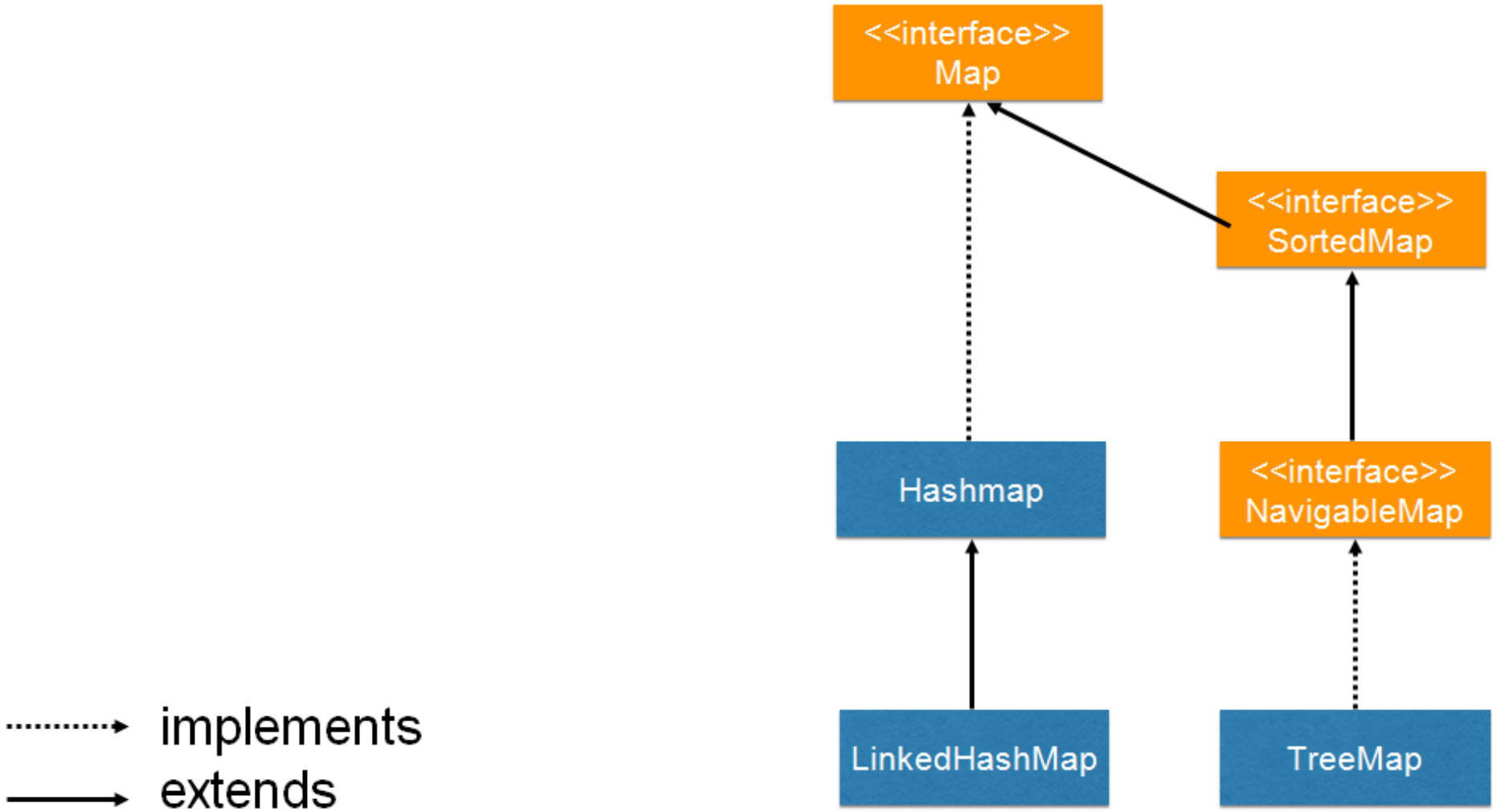
2. Reduces programming effort: The programmer need not to worry about design of Collection rather than he can focus on its best use in his program.

3. Increases program speed and quality: Increases performance by providing high-performance implementations of useful data structures and algorithms.

Hierarchy of Collection Framework


Iterable
Interface
Class
Collection
Implements
extends
List
Queue
Set
PriorityQueue
HashSet
ArrayList
Deque
LinkedHashSet
LinkedList
Vector
ArrayDeque
SortedSet
Stack
TreeSet

Map Interface



Core Interfaces in Collections

1. Collection :-> **Root interface** with basic methods like add(), remove(), contains(), isEmpty(), addAll(), ... etc.

2. List : Can contain duplicates and elements are ordered

The order in which elements are added is preserved in a List. This means if you insert elements in a certain sequence, they will appear in that same sequence when retrieved.

-> implementations are :

- LinkedList (linked list based)
- ArrayList(dynamic array based)

3. Set :->Doesn't allow duplicates. ((since it follows the Set behavior)

-> implementations of Set interface are:

- HashSet (Hashing based)
- TreeSet (balanced BST based).

Note that :->TreeSet implements SortedSet. which means that:

Elements are sorted in natural order (by default):

- Numbers → sorted in *ascending order*.
- Strings → sorted alphabetically (*lexicographically*).
- Custom Objects → can be sorted using *Comparable or Comparator*.

4. Queue : Typically order elements in FIFO order except like PriorityQueue.

5. Deque : Elements can be inserted and removed at both ends. Allows both LIFO and FIFO

6. Map : Contains Key value pairs. Doesn't allow duplicates.

Example :-> implementation HashMap and TreeMap. TreeMap implements SortedMap.

List Interface – Implementation classes

The ArrayList is a linear data structure, similar to the array, but resizable.

ArrayList (Pre JDK 1.5)

Prior to JDK 1.5, a collection is designed to hold java.lang.Object. Since Object is the Java's root class, all Java classes, which are descendants of Object, can be upcasted to Object and kept in a collection. However, when you retrieve an element from a collection (in the form of Object) it is the programmer's responsibility to downcast the Object back to its original class, before further manipulation can take place.

Below is an example of using an ArrayList to hold a collection of String objects.

Pre JDK 1.5 (RAW objects)

Class Program : 1

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListPreTest {

    public static void main(String[] args) {
        List lst = new ArrayList();
//  A List contains instances of Object. Upcast ArrayList to List

        lst.add("Java");
// add() takes Object. String upcast to Object implicitly

        lst.add("Luminar");
        lst.add("Technolab");
        //lst.add(20) //no compiler alert

            System.out.println(lst);     // [Java, Luminar, Technolab]

        Iterator iter = lst.iterator();
        while (iter.hasNext()) {      // any more element
             // Retrieve the next element, explicitly downcast from Object back to String
              String str = (String)iter.next();
              System.out.println(str);
        }
```

What is Iterator in Java?

An Iterator is an interface in Java (java.util.Iterator) that allows traversing elements in a collection one by one.

It was introduced in JDK 1.2 as a replacement for Enumeration.

Why Use Iterator?

- Works with modern collections (ArrayList, HashSet, LinkedList, etc.).
- Allows modifying a collection during iteration (supports remove()).
- Safer than Enumeration (avoids ConcurrentModificationException with fail-fast behavior)**

Method	Description
hasNext()	Returns true if there are more elements to iterate.
next()	Returns the next element.
remove()	Removes the last element returned by next().

Iterator iter = list.iterator();

list.iterator() - This calls the iterator() method on list, which is assumed to be a collection that implements the Iterable interface (e.g., ArrayList, LinkedList, HashSet, etc.). The iterator() method returns an Iterator object for iterating over the elements in the collection.

Iterator iter - This declares a variable iter of type Iterator. This will be used to traverse the elements of list.

Assignment (=) - Assigns the iterator obtained from list.iterator() to the variable iter

```
Iterator iter = list.iterator();

while (iter.hasNext()) {
// Retrieve the next element, explicitly downcast from Object back to String
                    String str = (String)iter.next();
                    System.out.println(str);
          }
```

Explanation of the above code:

List lst = new ArrayList(); // raw type

The list can accept any type of object because it uses a raw type (i.e., List without <Type>). This allows mixing different types like String, Integer, etc.

Later, during iteration:

This will throw ClassCastException at runtime if a non-String object (like 20) is in the list.

Concurrent Modification: Concurrent Modification in programming means to modify an object concurrently when another task is already running over it.

“Fail-fast” behavior in a collection refers to the strategy where an iterator immediately throws an exception (typically a ConcurrentModificationException) if the underlying collection is modified while it is being iterated over, effectively stopping the iteration process as soon as a structural change is detected and preventing potential inconsistencies in the data; this is a common feature in many Java collection implementations like ArrayList, HashSet, and HashMap. Structural modification means adding, removing any element from collection while a thread is iterating over that collection

Contrast with "fail-safe":

Fail-safe iterators: In contrast to fail-fast, a "fail-safe" iterator operates on a copy of the collection, meaning modifications to the original collection while iterating will not affect the iterator, as it is working on a snapshot of the data. This approach avoids throwing exceptions but might not reflect the latest state of the collection.

Eg:

```java
package collectionpack;


import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;


public class FailSafeFastIterators {
    public static void main(String[] args) {
        List<String> courses = new ArrayList<String>();
        courses.add("Java");
        courses.add("Spring");
        courses.add("Hibernate");
```

```java
Iterator<String> iterator = courses.iterator();

    while (iterator.hasNext()) {
        if (iterator.next().equals("Hibernate")) {
        iterator.remove(); // will not throw Exception
        ////If you remove an element via Iterator remove() method, exception will
        not be thrown.
        }
}
System.out.println(courses); //[Java, Spring]
}
}
```

Why Generics Were Introduced?

Generics were introduced in Java 5 to:

1. Provide Compile-Time Type Safety: adding wrong types to collections.
2. Eliminate Explicit Casting : Avoid (Type) casts when retrieving objects.
3. Improve Code Readability & Maintainability

Post JDK 1.5 (Generic)

Class Program : 2

ArrayList (Post JDK 1.5 – Generic- mention the Type if data in Collection)(Type Safety)

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListPostTest {
   public static void main(String[] args) {
      List<String> lst = new ArrayList<String>();  // Inform compiler about the
type
      lst.add("Java");         // compiler checks if argument's type is String
      lst.add("Luminar");
      lst.add("Technolab");
       //lst.add(20) //compiler alert
      System.out.println(lst);  // [Java, Luminar, Technolab]
```

```
 Iterator<String> iter = lst.iterator();   // Iterator of Strings
      while (iter.hasNext()) {
         String str = iter.next();  // compiler inserts downcast operator
         System.out.println(str);
      }

//      lst.add(new Integer(1234));   // ERROR: compiler can detect wrong type

//      Integer intObj = lst.get(0);  // ERROR: compiler can detect wrong type

      // Enhanced for-loop (JDK 1.5)
      for (String str : lst) {
         System.out.println(str);

      }
   }
}
```

Immutable Objects (String)

Class Program : 3

Using Enhanced for-loop on Collection of "Immutable" Objects (such as String)

```
import java.util.List;
import java.util.ArrayList;

public class ForEachImmutableTest {
   public static void main(String[] args) {
      List<String> lst = new ArrayList<String>();
      lst.add(“Java");
      lst.add("Luminar");
      lst.add("Technolab");
      System.out.println(lst);   // [Java, Luminar, Technolab]

      for (String str : lst) {
         str += "kochi!";   // cannot modify "immutable" objects
      }
      System.out.println(lst);   // [Java, Luminar, Technolab]
   }
}
```

mutable Objects (String)

Class Program : 4

Using Enhanced for-loop on Collection of "Mutable" Objects (such as StringBuilder)

```
import java.util.List;
import java.util.ArrayList;

public class ForEachMutableTest {
   public static void main(String[] args) {
      List<StringBuilder> lst = new ArrayList<StringBuilder>();
      lst.add(new StringBuilder(“Java"));
      lst.add(new StringBuilder("Luminar"));
      lst.add(new StringBuilder("Technolab"));
      System.out.println(lst);   // [Java, Luminar, Technolab]

      for (StringBuilder sb : lst) {
         sb.append("kochi");   // can modify "mutable" objects
      }
      System.out.println(lst);  // [Javakochi, Luminarkochi, technolgieskochi]
   }
}
```

Set Interface

Class Program : 5

Set Interface – Implementation

```
import java.util.*;

public class TestHashSet{
  public static void main( String[] args ){

                //Creating HashSet object

            HashSet<String> set = new HashSet<String>();

            //Adding elements to HashSet
            set.add("JAVA");
            set.add("JSP");
            set.add("STRUTS");
            set.add("HIBERNATE");
            set.add("JSP");   // adding duplicate
            set.add("JAVA"); // adding duplicate
```

```
        //Printing the elements of HashSet
        System.out.println(set);     //Output : [STRUTS, HIBERNATE, JSP, JAVA]


      //get the Enumeration object
      Enumeration e = Collections.enumeration(set);

      //enumerate through the HashSet elements

  while(e.hasMoreElements())
        System.out.println(e.nextElement());

      //You can notice that duplicate elements are not added to HashSet

    }

  }

}
```

What is Enumeration in Java?

Enumeration is an interface in Java (java.util.Enumeration) that provides a way to traverse elements of legacy collections like Vector and Stack.

It was introduced in JDK 1.0 and is considered an older alternative to Iterator.

The Enumeration interface provides two main methods:

- hasMoreElements() → Checks if there are more elements.
- nextElement() → Returns the next element.

Explanation of the above code:

Enumeration e -> This declares e as an Enumeration, which is an old-style Java interface for iterating over elements (before the introduction of Iterator in Java 2).

set is assumed to be a Collection (like HashSet, TreeSet, or any other Set implementation).

Collections.enumeration(set)

This method from the java.util.Collections class converts a Collection (in this case, set) into an Enumeration.

```
Enumeration e = Collections.enumeration(set);

        //enumerate through the HashSet elements

   while(e.hasMoreElements())
         System.out.println(e.nextElement());
```

Enumeration vs Iterator

Feature	Enumeration	Iterator
Introduced In	JDK 1.0	JDK 1.2
Applicable To	Legacy collections (Vector, Stack)	All Collections (List, Set, Map, etc.)
Methods	hasMoreElements(), nextElement()	hasNext(), next(), remove()
Supports Removal?	No	Yes (remove())
Preferred For	Read-only traversal	Modifiable collections

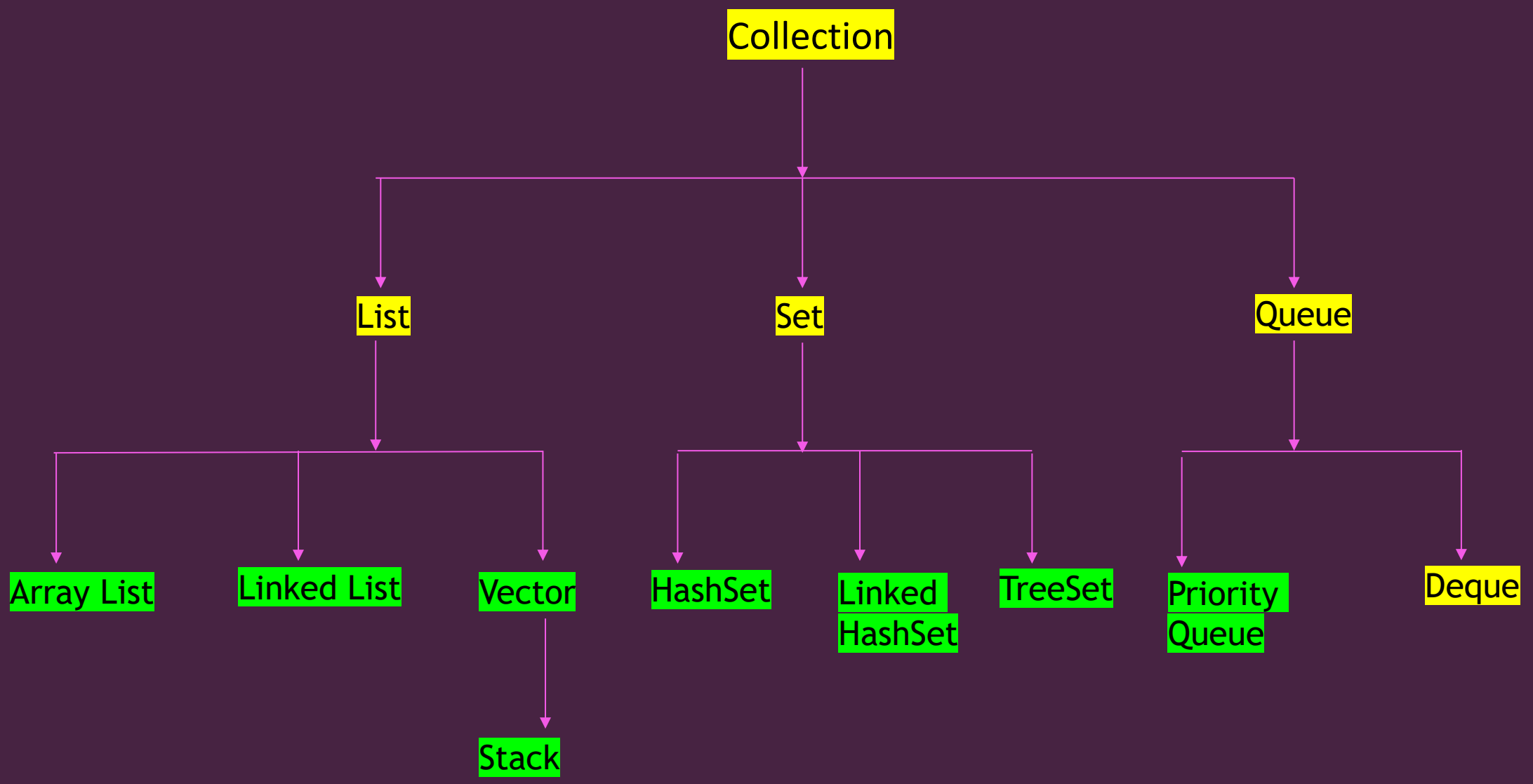
If Enumeration was created for legacy classes like Vector and Hashtable, then why is it still supported or usable with Set (and other collections)?

Because Java provides utility methods to create an Enumeration from any Collection, even if it doesn't natively use it.

Collections.enumeration() method.
This method wraps any Collection or Set into an Enumeration.

So even though Set does not natively return Enumeration, we can manually convert it to one using:

```
Enumeration e = Collections.enumeration(set);
```


Collection
List
Set
Queue
Array List
Linked List
Vector
Stack
HashSet
Linked HashSet
TreeSet
Priority Queue
Deque

Important List implementation classes

Class Program : 6

```java
package collectionpack;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Stack;
import java.util.Vector;

public class ListCollectionDemo {
   public static void main(String[] args) {

         List<Integer> listArray=new ArrayList<>();

         List<Integer> listLinked=new LinkedList<>();

         List<Integer> listVector=new Vector<>();

         List<Integer> listStack=new Stack<>();
```

```
listArray.add(11);
listArray.add(22);
listArray.add(33);

listLinked.add(11);
listLinked.add(22);
listLinked.add(33);

listVector.add(11);
listVector.add(22);
listVector.add(33);

listStack.add(11);
listStack.add(22);
listStack.add(33);
```

```java
//printing/fetching/traversing/iteration on collections
// print as insertion order
System.out.println("arraylist values \n==================================\n");
for(Integer i: listArray)
System.out.println(i);

System.out.println("\nlinked list values
\n==================================\n");
for(Integer i: listLinked)
System.out.println(i);

System.out.println("\nvector values \n==================================\n");
for(Integer i: listVector)
System.out.println(i);

System.out.println("\nstack values \n==================================\n");
for(Integer i: listStack)
System.out.println(i);
}
}
```

```java
List<Integer> listArray=new ArrayList<>();

List<Integer> listLinked=new LinkedList<>();

List<Integer> listVector=new Vector<>();

List<Integer> listStack=new Stack<>();
```

These four lines of code create lists that implement the `List<Integer>` interface in Java, but they differ in their underlying implementations and behavior.

What is Time Complexity and Space Complexity?

In Data Structures and Algorithms (DSA) — and especially in Java programming — we analyze how efficient our code is using:

Time Complexity : Speed/performance as input grows

Space Complexity : Memory usage as input grows

Measured in Big-O notation (e.g., O(n), O(log n))

What is Big O Notation?

Big O Notation is a mathematical way to describe how the time or space requirements of an algorithm grow as the size of the input (n) increases. (ie. Describes the growth rate of algorithm)

Focus : On input size vs operations/memory

Use : To compare efficiency of algorithms

1. List<Integer> listArray=new ArrayList<>();

Implementation: Backed by a dynamic array (ie. ArrayList).

Performance:

- Fast random access (`O(1)` time complexity) since it is an array
- Insertion and deletion from the end are usually fast (`O(1)`), but inserting or deleting from the middle requires shifting elements (`O(n)`).

Use Case: Best for frequent access operations, such as getting elements by index.

Insertion and deletion from the end are usually fast (`O(1)`), but inserting or deleting from the middle requires shifting elements (`O(n)`).

Insertion/Deletion at the End → O(1) (Constant Time)

O(1) means that the time taken by an operation is constant, regardless of the size of the input.(No matter if your array has 10 elements or 10 million, the operation takes the same amount of time).

Eg:

```
int[] arr = {10, 20, 30};
```

If you want to insert at the end,

e.g., arr[3] = 40, it's just one operation(Just assign at the next index)

Same for deletion

No need to shift elements. That's why it's O(1).

Insertion/Deletion in the Middle → O(n) (Linear Time)

Now suppose you want to insert 25 between 20 and 30:

arr = {10, 20, _, 30,40}; // make space at index 2
You must shift 30 and 40 one index to the right to make space:

If there are n elements and you insert at index k, you may have to shift n - k elements.

Eg. In the above example no of elements(n)=4, index k=2, to shift 4-2=2 elements

That's why inserting or deleting from the middle is O(n) — shifting costs time.

Only 2 elements were shifted — shouldn't that be O(2)?
No — Big O doesn't work that way.O(2) . because constants (like 2, 3 , etc) are ignored in Big O notation. We use O(n) to describe how the number of operations grows in proportion to input size. and even if it's just 2 elements today, it could be n elements tomorrow.

ArrayList is best for general-purpose use where random access is needed.

LinkedList excels when frequent insertions/deletions from either end of the list are required.

Vector is similar to `ArrayList` but is thread-safe.

Stack is a `Vector` with additional methods for stack operations, useful when LIFO behavior is needed.

Important Set implementation classes

Class Program : 7

```java
package collectionpack;


import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;


public class SetCollectionDemo {


public static void main(String[] args) {
      Set<Integer> setHash = new HashSet<>();
      Set<Integer> setLinked = new LinkedHashSet<>();
      Set<Integer> setTree = new TreeSet<>();
```

```java
setHash.add(23);
setHash.add(79);
setHash.add(6);
setHash.add(14);
setHash.add(75);
setHash.add(2);

System.out.println("setHash"); //any order
for (Integer h : setHash) {
   System.out.println(h);
}
```

```java
setLinked.add(23);
setLinked.add(79);
setLinked.add(6);
setLinked.add(14);
setLinked.add(75);
setLinked.add(2);
System.out.println("setLinked"); //insertion order
   for(Integer l:setLinked){
System.out.println(l);
}

}
```

```java
setTree.add(23);
setTree.add(79);
setTree.add(6);
setTree.add(14);
setTree.add(75);
setTree.add(2);
System.out.println("setTree"); //sorted order
for(Integer t:setTree){
System.out.println(t);
}




}
```

any order

```
setHash
2
6
23
75
14
79
```

insertion order

```
setLinked
23
79
6
14
75
2
```

sorted order

```
setTree
2
6
14
23
75
79
```

Thank you ☺ Happy learning ☺